

Workshop: Strapi + Vite, in one class period

A hands-on lab pairing with slides 15–18 of the Headless CMS deck.

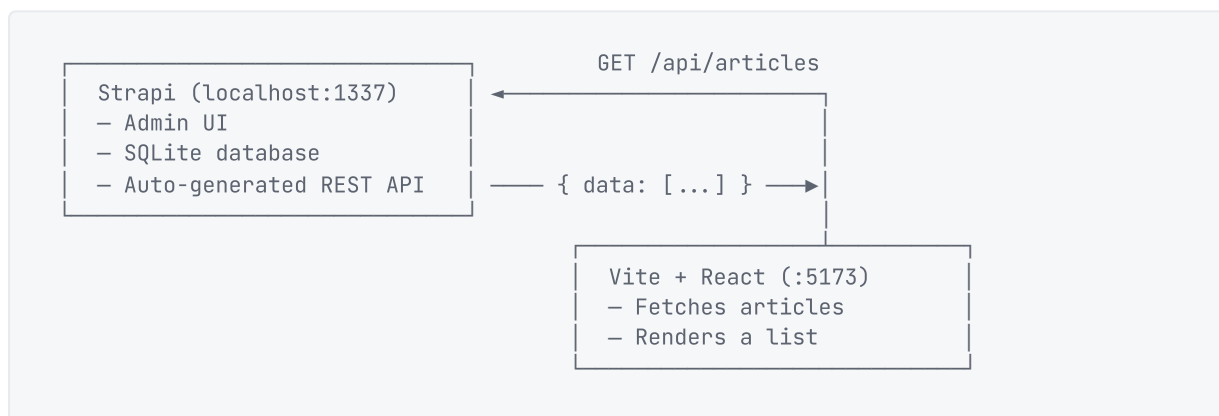
Duration: ~80 min (60 min minimum)

Stack: Strapi 5 + Vite + React

Outcome: Live API → React render

§ What we're building

A tiny *course notes* page. Strapi stores the content. A Vite + React app fetches it and renders it. That's the entire scope of the class — we are deliberately not building a complete blog.



By the end the student should be able to:

1. Create content in Strapi's admin panel and have it appear in a React app without redeploying anything.
2. Identify the boundary between *backend serves data* and *frontend renders data* — the central headless-CMS idea.
3. Recognize and fix the three or four things that always break the first time.

A NOTE ON TEMPLATES

Strapi maintains official starters for Gatsby, Next.js, and Nuxt — but not Vite. We use the standard `npm create vite@latest` template (the de-facto React starter) and wire it to Strapi by hand. This takes 5 extra minutes and is worth it: students see the integration explicitly rather than inheriting it.

§ Before class — student prerequisites

Send this list out the day before. If a student arrives with none of it installed, the class will be spent on Node installation and not on Strapi.

- **Node.js 20 LTS or 22 LTS** — verify with `node --version`. Strapi 5 requires an active LTS release; odd-number versions (19, 21, 23) are not supported.
- **A code editor** — VS Code is fine.
- **A terminal they can use** — built-in macOS/Linux terminal, or Windows Terminal / Git Bash on Windows.
- **About 600 MB of disk space** for the `node_modules`.

01 Stand up Strapi — 15 min

1.1 Create the project

In a terminal, in a folder where you want the project to live:

BASH

```
$ npx create-strapi@latest blog-cms --quickstart --skip-cloud
```

What each piece does:

PIECE	WHAT IT DOES
<code>npx create-strapi@latest</code>	Downloads and runs the Strapi installer. (<code>create-strapi-app@latest</code> also still works — same thing.)
<code>blog-cms</code>	The folder name for your project.
<code>--quickstart</code>	Use SQLite (no separate database to install).
<code>--skip-cloud</code>	Skip the Strapi Cloud login prompt — we're staying local.

The installer will take 1–3 minutes. When it finishes, Strapi auto-launches and opens `http://localhost:1337/admin` in your browser.

1.2 Create the first admin user

The browser shows a registration form. Fill it in (it's local, the password doesn't have to be real). Click **Let's start**.

COMMON PITFALL

If the browser doesn't open, look at the terminal. You'll see *"Welcome back! To manage your project, go to the administration panel..."* with the URL. Just visit it manually.

You should now be staring at the Strapi admin dashboard. Congratulations — you have a CMS running on your laptop.

1.3 Confirm the server keeps running

Leave this terminal alone for the rest of the workshop. If you ever need to restart Strapi:

BASH

```
$ cd blog-cms  
$ npm run develop
```

02 Build a content type – 15 min

This is where headless CMS thinking starts. We're going to define the *shape* of an article, then create some.

2.1 Open the Content-Type Builder

In the left sidebar of the Strapi admin: **Content-Type Builder**.

Click **Create new collection type**.

- **Display name:**
- Click **Continue**.

Strapi will ask you to add fields one at a time. Add these four:

FIELD	TYPE	NOTES
<code>title</code>	Text (Short text)	The article headline
<code>slug</code>	UID	"Attached field": <code>title</code> . Auto-generates a URL-safe slug.
<code>body</code>	Text (Long text)	Plain text — rich text adds rendering complexity.
<code>excerpt</code>	Text (Short text)	A 1-2 sentence summary

After adding the fourth field, click **Save** (top right). Strapi will say *"Server is being restarted..."* — wait for it (5–10 seconds). The auto-restart applies the new schema.

WHY UID, NOT TEXT?

UID fields are unique and auto-format. They prevent duplicate slugs and convert *"My First Article"* to `my-first-article` automatically. This is a small lesson in why CMSs have specialized field types.

2.2 Create three articles

In the sidebar: **Content Manager** → **Article**. Click + **Create new entry**. Fill in:

FORM

Title: Welcome to the course
 Slug: (click the refresh icon — auto-fills as "welcome-to-the-course")
 Excerpt: A quick orientation to what we'll cover this term.
 Body: (any few sentences)

Click **Save**, then click **Publish**. Repeat twice more with whatever titles you like.

CRITICAL

Save and **Publish** are different actions. Save creates a draft. Publish makes it queryable via the public API. If you only Save, the API will return an empty array and you'll spend 20 minutes wondering why.

2.3 Open the door to the API

Even with three published articles, the API is still locked down. By default, Strapi grants nothing to the public.

- Sidebar: **Settings** → **Users & Permissions plugin** → **Roles**.
- Click **Public**.
- Scroll to **Article** in the permissions list.

- Tick the boxes for `find` and `findOne` .
- Click **Save** (top right).

THE SINGLE MOST COMMON 403 IN THIS LAB

Strapi *creates* content openly — but *publishing* it is a separate decision from *exposing* it. If you skip this step, every API call will return `403 Forbidden` .

03 First API call – 5 min

Open a new browser tab and go to:

URL

`http://localhost:1337/api/articles`

You should see JSON like this (Strapi 5 — flat shape):

JSON

```
{
  "data": [
    {
      "id": 1,
      "documentId": "abc123def456",
      "title": "Welcome to the course",
      "slug": "welcome-to-the-course",
      "excerpt": "A quick orientation to what we'll cover this term.",
      "body": "...",
      "createdAt": "2026-05-07T...",
      "publishedAt": "2026-05-07T...",
      "locale": "en"
    }
  ],
  "meta": {
    "pagination": { "page": 1, "pageSize": 25, "total": 3 }
  }
}
```

Pause for a moment and have students inspect this. Three things to point out:

1. `data` is an **array** of articles. Each article is a flat object — fields like `title` are at the top level, not nested under `attributes` . (This was different in Strapi v4. Old tutorials show `data.attributes.title` — that syntax no longer works in Strapi 5 unless you opt in with a special header.)

2. `documentId` is the new stable identifier in Strapi 5. Numeric `id` is still there but `documentId` is what you'll use long-term.
3. `meta.pagination` is automatic. You didn't write this — Strapi generated it.

04 Vite + React frontend – 30 min

4.1 Scaffold the app

Open a new terminal window (leave Strapi running in the first one). Navigate to a folder *outside* `blog-cms`:

BASH

```
$ npm create vite@latest blog-frontend -- --template react
$ cd blog-frontend
$ npm install
$ npm run dev
```

A new tab opens on `http://localhost:5173` showing the Vite + React starter page. Working frontend confirmed.

4.2 Set the Strapi URL

We need to tell the frontend where Strapi lives. In the `blog-frontend` folder, create a file called `.env` (yes, just `.env`, no other extension):

.ENV

```
VITE_STRAPI_URL=http://localhost:1337
```

VITE-SPECIFIC GOTCHA

Environment variables that should reach the browser must start with `VITE_`. Anything else stays server-side. Access them in code via `import.meta.env.VITE_STRAPI_URL` — **not** `process.env`. This trips up students coming from Create React App or Next.js.

After saving `.env`, **stop the dev server** (`Ctrl+C`) and **start it again** (`npm run dev`). Vite only reads env files on startup.

4.3 Replace App.jsx

Open `src/App.jsx`. Delete everything in it. Paste this:

SRC/APP.JSX

```
import { useEffect, useState } from 'react'
import './App.css'

const STRAPI_URL = import.meta.env.VITE_STRAPI_URL

function App() {
  const [articles, setArticles] = useState([])
  const [error, setError] = useState(null)
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    fetch(`${STRAPI_URL}/api/articles`)
      .then((res) => {
        if (!res.ok) throw new Error(`Strapi responded ${res.status}`)
        return res.json()
      })
      .then((json) => {
        setArticles(json.data)
        setLoading(false)
      })
      .catch((err) => {
        setError(err.message)
        setLoading(false)
      })
  }, [])

  if (loading) return <p>Loading...</p>
  if (error) return <p style={{ color: 'crimson' }}>Error: {error}</p>

  return (
    <main>
      <h1>Course Notes</h1>
      {articles.map((article) => (
        <article key={article.documentId}>
          <h2>{article.title}</h2>
          <p><em>{article.excerpt}</em></p>
          <p>{article.body}</p>
        </article>
      ))}
    </main>
  )
}

export default App
```

Save the file. The browser at `:5173` should hot-reload and show your three articles.

WALK THROUGH THE CODE WITH THE CLASS

Spend 3–4 minutes here. The whole point of the lab.

LINE / BLOCK	WHAT IT TEACHES
<code>import.meta.env.VITE_STRAPI_URL</code>	How Vite exposes env vars.
<code>fetch(...)</code>	Plain browser API — no library needed.
<code>if (!res.ok) throw...</code>	Why response status matters; without this, a 403 looks like "no articles."
<code>json.data</code>	The Strapi 5 response shape — array of flat objects.
<code>key={article.documentId}</code>	React keys; use the stable identifier, not array index.
<code>article.title</code> , <code>article.body</code>	Direct field access — the v5 flat shape.

4.4 A tiny bit of style

Open `src/App.css`. Replace its contents:

```

SRC/APP.CSS

main {
  max-width: 700px;
  margin: 3rem auto;
  padding: 0 1.5rem;
  font-family: system-ui, sans-serif;
  line-height: 1.6;
  color: #222;
}

h1 {
  font-size: 2.5rem;
  margin-bottom: 2rem;
  border-bottom: 2px solid #eee;
  padding-bottom: 0.5rem;
}

article {
  margin-bottom: 2.5rem;
  padding-bottom: 1.5rem;
  border-bottom: 1px solid #f0f0f0;
}

article h2 { margin-bottom: 0.3rem; }
article em { color: #666; }

```

That's it for the core lab. **What you have now:** a Strapi backend + a React frontend pulling articles from it and rendering them. Add a new article in the Strapi admin, refresh the browser, watch it appear. The headless workflow, working.

§ Checkpoint — what students should have running

Two terminals, two browser tabs, one CMS, one frontend, three articles.

	PROCESS	URL	WHAT'S THERE
Terminal 1	<code>npm run develop</code> (in <code>blog-cms</code>)	<code>localhost:1337/admin</code>	Strapi admin with 3 articles
Terminal 2	<code>npm run dev</code> (in <code>blog-frontend</code>)	<code>localhost:5173</code>	React app rendering them
Bonus tab	(browser)	<code>localhost:1337/api/articles</code>	Raw JSON from the API

If everyone has these, the lab has succeeded. Stop here if time is tight.

05 Stretch tasks — 10 min, optional

Pick one. Each takes ~10 minutes and surfaces a real headless concern.

Stretch A — Sort newest first

Change the fetch URL in `App.jsx`:

SRC/APP.JSX

```
fetch(`${STRAPI_URL}/api/articles?sort=publishedAt:desc`)
```

Add a fourth article in Strapi with a recent timestamp. Watch it appear at the top.

LESSON

Sorting happens server-side, declared via URL parameters. The frontend stays unchanged. This is the headless equivalent of a SQL `ORDER BY` — you don't sort in JavaScript.

Stretch B — Add a cover image

1. Back in Strapi: **Content-Type Builder** → **Article** → **Add another field**.
2. Choose **Media** → **Single media**, name it `cover`, click **Add this field** → **Save**.
3. Edit one of your articles, upload an image, **Save**, **Publish**.
4. Update the fetch URL — relations don't populate by default in Strapi 5:

SRC/APP.JSX

```
fetch(`${STRAPI_URL}/api/articles?populate=cover`)
```

Render it (in the JSX inside the `articles.map`):

SRC/APP.JSX

```
{article.cover && (  
  <img  
    src={` ${STRAPI_URL}${article.cover.url}`}  
    alt={article.title}  
    style={{ maxWidth: '100%' }}  
  />  
)}
```

LESSON

Strapi returns relative URLs. The frontend must prepend the Strapi host. In production, you'd configure Strapi to serve via a CDN. Also: `?populate=cover` is essential — without it, the relation is silently empty.

Stretch C — A second content type with a relation

1. In Strapi, create a new collection type **Author** with fields: `name` (Text), `bio` (Text).
2. Add an **Author** to the Article schema as a **Relation** field — *Article has one Author*.
3. Create one or two authors.
4. Edit each Article and assign an Author.
5. **Don't forget:** Settings → Roles → Public → enable `find` and `findOne` on **Author** too.
6. Update the fetch:

SRC/APP.JSX

```
fetch(`${STRAPI_URL}/api/articles?populate=author`)
```

Render the byline:

SRC/APP.JSX

```
{article.author && <p style={{ color: '#888' }}>By {article.author.name}</p>}
```

LESSON

Content modeling is the heart of CMS work. A relation in Strapi is a foreign key in the database, plus admin UI, plus auto-generated API endpoints, plus populate machinery — all from clicking a checkbox. This is what you're paying for.

§ What can go wrong (and how to fix it)

A short troubleshooting reference. Print this on the back of the handout.

SYMPTOM	MOST LIKELY CAUSE	FIX
<code>403 Forbidden</code> from <code>/api/articles</code>	Public role doesn't have <code>find</code> permission	Settings → Roles → Public → tick <code>find</code> and <code>findOne</code> on Article
<code>data: []</code> (empty array) but Strapi has articles	Articles aren't published, only saved as drafts	Open each article in Content Manager → click Publish
CORS error in browser console	Frontend on a different origin than localhost (rare in dev)	Strapi 5 allows localhost by default; restart Strapi if needed
<code>import.meta.env.VITE_STRAPI_URL</code> is <code>undefined</code>	<code>.env</code> was created after <code>npm run dev</code> started	Stop dev server (<code>Ctrl+C</code>), run <code>npm run dev</code> again
Cover image is <code>null</code> even after uploading	Forgot <code>?populate=cover</code> in the fetch URL	Add it. Strapi 5 doesn't auto-populate relations
<code>a.attributes.title</code> is <code>undefined</code>	You're using Strapi v4 syntax in a v5 project	Switch to <code>a.title</code> — the response is flat in v5
Strapi terminal shows port already in use	Old Strapi still running from a previous session	Kill it: <code>pkill -f strapi</code> (Mac/Linux) or close the terminal

§ Reflection prompts

For the last 5 min, or homework. Pick two to discuss as a class:

1. *In one sentence: what would happen to your React app if Strapi crashed?*
(The point: separation. The frontend would break, but the content is safe in the database.)
2. *Imagine you also need to display these articles in a mobile app. What would you need to change in Strapi? What would you need to build new?*
(Strapi: nothing. Build new: a mobile client that hits the same API.)
3. *Why did we have to manually enable the Public role permissions? What's the design reasoning?*
(Default-deny. The opposite is the WordPress default of "everything public unless you lock it down" — which has caused a lot of breaches.)
4. *Strapi 5 changed the API response shape from `data.attributes.title` to `data.title`. If you had a live app on v4 and migrated to v5, what would break?*
(Every component that reads from the API.)

§ Files the student will leave with

FILESYSTEM

```
blog-cms/                                ← Strapi backend
├─ src/api/article/                      ← schema generated from the admin UI
├─ database/                            ← SQLite DB lives here
├─ .tmp/data.db                         ← actual data file
└─ ...

blog-frontend/                          ← Vite + React
├─ src/App.jsx                          ← the only file we wrote in
├─ src/App.css                          ← the only file we styled
├─ .env                                ← VITE_STRAPI_URL
└─ ...
```

If they zip and submit both folders, you can run the project on any other machine by `npm install` in each folder, then `npm run develop` and `npm run dev` in two terminals. The SQLite DB file inside `blog-cms/.tmp/` carries their content with it.

§ Notes for the instructor

1. **Strapi 5 vs Strapi 4 syntax in the lecture deck.** The presentation slide 17 currently shows `a.attributes.title` (v4 syntax). For Strapi 5 the correct access is `a.title`. Update the slide before delivery — or set the response-format header in the fetch:

JS

```
fetch(url, { headers: { 'Strapi-Response-Format': 'v4' } })
```

This is a temporary back-compat header Strapi provides for migration. I recommend updating the slide instead — v5 is current and the flat shape is genuinely simpler.

2. **The Strapi admin login screen on first run.** The installer may briefly prompt for a Strapi Cloud login. The `--skip-cloud` flag in the command above bypasses this. Without it, students will get confused by a sign-up screen they don't need.
3. **Why no router?** The presentation lab spec used `react-router-dom` for a list+detail pattern. This workshop intentionally simplifies to a single list view. The router can be added in 10 minutes if you have time, but the headless concept lands without it.
4. **Why no styling library?** Same reason. Adding Tailwind or any UI kit forks the lesson into *"how does this UI library work?"* The 30 lines of CSS in `App.css` are enough to make the result not embarrassing.
5. **What if a student is on Windows and `npx` hangs?** The `create-strapi` install runs faster on Mac and Linux. On Windows, allow up to 5 minutes the first time. If it fails outright, switching to `npm create strapi@latest blog-cms -- --quickstart --skip-cloud` usually works.
6. **Pre-flight the lab.** Run the entire workshop on a fresh machine the day before. Strapi releases minor versions frequently and occasionally something in the wizard changes. A 10-minute pre-flight catches anything new.